VisionX - Home Security System

Problem

Home security cameras are expensive, especially with subscriptions, and are difficult to customize. I want a system that allows me to monitor cameras, save footage locally to my NAS, and add custom automation features like facial recognition without relying on third-party services.

Many security camera subscription services make it difficult to cancel the subscription, so I have just been getting charged for simple camera functions that aren't fully used. I wanted to build something that can be run on your own device and is scaled based on available hardware, making it fully customizable and cheap.

Overview

This project is a self-hosted home security camera platform with a web dashboard and mobile app for viewing live camera feeds, storing footage on a NAS, and monitoring device health. This system will eventually support motion detection, facial recognition, alerts, mmWave heartbeat detection, and customizable automations.

Goal

The goal is to build a working MVP that can connect to at least one IP camera, display a live feed in a web dashboard, save recordings to my NAS, and show basic camera status such as online/offline health.

The final goal is a self-hosted edge AI home security system that uses IP camera footage connected to a PoE switch and provides a stream for an edge-inference device (Optiplex 8500T) to record camera footage, detect meaningful events (package arrival, suspicious car parked, family member leaving house, etc.), connects with Home Assistant, and lets users ask natural-language questions about what happened at home

Ex.

Instead of: “Can I see my front door camera”

You can ask: “When did Dad leave home”

Target Users

The first user is myself, for monitoring my home. Future users could include people who want a customizable, self-hosted alternative to a subscription-based security service

MVP Scope

- Web dashboard to view live camera feeds
- Footage stored on NAS
- Device health monitoring
- Basic authentication for the dashboard
- Add or manage one or more camera streams

Out of Scope for MVP

- Mobile app
- Motion Detection
- Facial Recognition
- Notification Rules
- Multi-user permissions
- Cloud backup

Available Devices

- Pi 5 with Hailo Inference Chip
- Arducam
- Dell Optiplex 8500T 8GB RAM
- mmWave sensors
- ESP32

Success Criteria

- I can open the dashboard and view the live camera feed
- System detects when camera goes offline
- Recordings saved to NAS
- App can run continuously on homelab using docker

Tech Stack

- Frontend Web: React + Vite + Tailwind CSS
- Backend API: FastAPI + Python
- Database: PostgreSQL
- Live Streaming: go2rtc
- Recording: FFmpeg
- Storage: NAS mounted into backend/worker containers
- Background Jobs: Redis + Python workers
- Deployment: Docker Compose
- Reverse Proxy: Caddy
- Remote Access: Tailscale
- Monitoring: Prometheus + Grafana
- Mobile: PWA first, React Native/Expo later
- AI: OpenCV first, ONNX/TensorFlow Lite later

Database

- Users
- Cameras
- Recordings (file path)
- Motion events
- Face detections
- Recognized people
- Device health
- Alerts
- Automation rules
- Camera status logs

Ex.

recording_id: 123
camera_id: front_door
start_time: 2026-06-27 10:00:00
end_time: 2026-06-27 10:05:00
file_path: /mnt/nas/security/front_door/2026-06-27/10-00.mp4
has_motion: true
has_person: true

Tradeoffs

- FastAPI because AI/video processing mainly uses python

Considerations

Camera Streaming: Frigate vs go2rtc vs build myself

Frigate

- Open source local NVR designed for IP cameras with AI object detection
- Uses OpenCV and TensorFlow for local object detection
- Recommends using a dedicated detector instead of CPU detection
- Supports:
    - Camera streams, recording, snapshots, object detection, MQTT events, Home Assistant integration, Retention rules, local processing
- Drawbacks:
    - Custom facial recognition may require a separate service
    - App may depend heavily on Frigate's design

Go2rtc

- Takes camera streams and restream through RTSP/webRTC
- Helps with low-latency live viewing and reducing direct connections to cameras
- Drawbacks:
    - Not full NVR by itself
    - Need to build recording, detection, events, storage, and UI

Build everything with FFmpeg/OpenCV

- Pros:

        - Full control recording, detection, and storage logic
- Drawbacks:
    - Video streaming difficult to program
    - RTSP issues are common
    - Browser live playback is not trivial
    - Recording reliability is hard
    - Mobile streaming becomes more difficult to implement

Database

TimescaleDB

- PostgreSQL extension for time-series data through hypertables, partitioned by time for analytics workloads
- Useful if storing lots of camera health logs, motion counts, object counts, temperature readings, heartbeat data, or metrics over time

Mobile App

Progressive Web App (PWA)

- Build dashboard as mobile-friendly web app, host it on homelab, and open in Safari and add to homescreen
- Pros:
    - No app store or apple developer account required
    - No testflight
    - No native IOS signing
    - Easy to update
- Drawbacks:
    - Not as native-feeling as real IOS app
    - Limited access to iphone APIs
    - Push notifications are more complicated than native apps
    - Background tasks are limited

Expo / React Native

- Pros:
    - Real app
    - Better native navigation

    - Better access to device APIs
    - Better push notification
- Drawbacks:
    - More setup and requires Apple signing/provisioning
    - Requires paid Apple Developer account
    - More difficult updates
    - Separate mobile codebase from web dashboard

Architecture

PoE Camera / Pi Camera / Sensors

↓

RTSP / MQTT / Home Assistant Webhooks

↓

VisionX Ingestion Layer

↓

Recording worker + AI Worker + Event Worker

↓

PostgreSQL event timeline

↓

NAS video clips / snapshots

↓

FastAPI

↓

React/PWA Dashboard + Home Assistant + Chat Assistant

Edge Inference Device acts as edge server. It receives streams, record footages, runs lightweight inference, stores metadata, and coordinates workers

Home Assistant Integration:

Home assistant feeds VisionX extra context through **MQTT** or **webhooks**

Ex.

Front_door opened

Garage opened

Dad's phone left home

Living room motion detected

Driveway camera saw vehicle

Decision:

Dad phone left home + front door opened + familiar person detected + drivecar car left = Dad left home

Milestones

Phase 1: Build MVP

Phase 1.1: Research camera streaming, tech stacks, RTSP

Goal: understand camera pipeline before building

- Pick camera source: webcam first, then PoE camera later
- Confirm RTSP stream works in VLC
- Run go2rtc locally
- Confirm go2rtc can restream camera
- Confirm browser can view live stream
- Decide folder structure
- Create Docker Compose Base

Camera/Webcam -> RTSP -> go2rtc -> browser

Phase 1.2: Backend foundation

Goal: Create API and database foundation

- FastAPI app setup
- PostgreSQL connection
- SQLAlchemy models
- Alembic migrations
- Camera model
- Recording model
- Health check model
- Snapshot/event model placeholders
- Pydantic schemas
- Basic error handling

API Endpoint:

GET /api/v1/health

POST /api/v1/cameras

GET /api/v1/cameras

GET /api/v1/cameras/{id}

PATCH /api/v1/cameras/{id}

DELETE /api/v1/cameras/{id}

POST /api/v1/cameras/{id}/test-connection

Phase 1.3: Camera connection prototype

Goal: prove VisionX can communicate with real camera stream

- Store RTSP URL in database
- Test stream with FFmpeg/OpenCV
- Check if camera is online/offline
- Save latest health check to database
- Return camera status from API

Phase 1.4: Web dashboard live view

Goal: View cameras from browser

- Login page placeholder
- Dashboard layout
- Camera grid page
- Single camera detail page
- Camera add/edit form
- Live video player component
- Camera status badge

Frontend API calls:

- fetch cameras
- add camera
- edit camera
- delete camera
- test camera connection

Phase 1.5: Recording system

Goal: save footage to local/NAS storage

- Recording worker
- FFmpeg recording command
- Segment recordings into 5-minute clips
- Save files to /recordings/{camera}/{date}/
- Create recording row in PostgreSQL
- Update recording status when finished
- Store file path, start time, end time, duration
- Generate thumbnail/snapshot

API Endpoints:

GET /api/v1/recordings

GET /api/v1/recordings/{id}

GET /api/v1/cameras/{id}/recordings

Phase 1.6: NAS storage

Goal: Make recordings save to NAS, not just local disk

- Mount NAS folder on host
- Pass NAS folder into Docker container
- Confirm worker can write files
- Confirm API can read metadata
- Confirm frontend can play/download clips

Phase 1.7: Background jobs

Goal: move long-running work out of API

- Redis container
- Worker container
- Background job structure
- Camera health check job
- Recording job
- Thumbnail job
- Cleanup/retention job placeholder

Phase 1.8: Basic Authentication

Goal: protect dashboard

- User table
- Password hashing
- Login endpoint
- JWT/session auth
- Protected API routes
- Frontend login flow
- Logout button

Phase 1.9: Docker deployment

Goal: run everything as a homelab service

Services:

- visionx-web
- visionx-api

- visionx-worker
- visionx-postgres
- visionx-redis
- visionx-go2rtc
- Visionx-caddy

Build:

- docker-compose.yml
- environment variables
- volumes for database/storage
- Caddy reverse proxy
- local domain or Tailscale access

Phase 1.10: MVP polish

Goal: make it portfolio-ready

- Clean README
- Architecture diagram
- Setup instructions
- Screenshots/GIF demo
- API documentation
- Database ERD
- Known limitations
- Future roadmap
- Resume bullet points

Phase 2: Smart Event Engine

Teach VisionX what happened

- Motion events
- Person detection
- Vehicle detection
- Package detection
- Snapshots
- Event timeline
- Event filtering
- Event playback

Phase 3: Intelligent Rules

Convert detections into meaningful events

Examples

- Package delivered
- Package removed
- Vehicle arrived
- Vehicle left
- Person loitering
- Unknown visitor
- Zone intrusion
- Garage opened

Phase 4: Home Assistant Integration

VisionX becomes part of your smart home

- MQTT
- Webhooks
- Entity synchronization
- Door sensors
- Garage
- Smart locks
- Presence detection
- ESP32 devices

Phase 5: Indoor AI nodes

Example
Pi 5
Camera
mmWave
Microphone
Speaker
Temperature
GPIO

Possible features

- Occupancy
- Fall detection
- Heartbeat
- Sleep monitoring
- Room activity
- Voice commands

Phase 6: Natural Language Search

Instead of searching video manually:

"When did Dad leave?"
"When was the package delivered?"
"Show everyone who entered the garage yesterday."
"What happened while I was at work?"

VisionX searches the event timeline.

Phase 7: AI Summaries

Daily reports

Today

• Amazon delivered one package.
• Dad left at 8:14 AM.
• Mail arrived.
• Neighbor walked dog.
• Unknown car parked for 2 minutes.
• No suspicious activity.

Phase 8: Contextual AI Assistant

User:
Did anyone come over?

VisionX:
Yes.
Sarah visited from 2:13 PM to 3:40 PM.

The front door opened twice.
She left through the garage.
Would you like to watch the clip?

Phase 9: Multi-Node VisionX

Supports multiple AI devices

Phase 10: Mobile App

Native app

- Notifications
- Live view
- Timeline
- Search
- Smart alerts
- AI assistant

Fun Possible Additions

- Object tracking for intruders